MultiHopBench: A Constraint-Aware Benchmark for Multi-Hop and Multi-Constraint Factual Question Answering

Matvei Borodin
HSE University
mvborodin@edu.hse.ru

Abstract

Factual question-answering benchmarks often reduce correctness to a short exact-match target, obscuring failures in inverse retrieval, list completion, simultaneous constraint satisfaction, and no-answer recognition. We introduce MultiHopBench, a benchmark that links natural-language requests to structured constraints and formally verifiable answers. Its Wikidata-derived core is complemented by curated external examples, validated zero-gold near misses, and parallel Russian formulations. The evaluation architecture separates atomic LLM-based semantic annotation from deterministic schema validation and score recomputation. We demonstrate the reporting workflow with a compact study of eight named systems, 14 observed model–setting configurations, two languages, and 1,300 queries per language, totaling 36,400 responses. Missing settings are reported as `N/A` and never imputed. In the only complete factorial design, `GPT-5.4-mini` scores 60.7 in memory-only, 81.0 in search-only, 78.2 in reasoning-only, and 92.0 with search + reasoning. Across four paired memory-to-reasoning comparisons, reasoning adds 17.5–23.6 points; `Qwen3.5-397B-A17B` leads the observed memory-only results, while `DeepSeek-V3.2` leads the observed reasoning-only results.

1 Introduction

Large language models (LLMs) are increasingly used as factual interfaces in search, dialogue, and analytical systems. In practice, users rarely ask only for one isolated fact. They ask for several entities that jointly satisfy a set of conditions, refer to relations in an inverse direction, combine properties of linked objects, impose temporal or numerical thresholds, and sometimes formulate requests whose conditions are mutually incompatible. Such requests are closer to database queries expressed in natural language than to conventional single-answer QA. A response can therefore be fluent and topically plausible while silently violating a country, period, relation, object type, negation, or requested list size.

This setting is related to the reversal curse: knowledge that is accessible in one direction is not necessarily retrieved from an inverse formulation (Berglund et al., 2023). Our target is broader than reproducing a paired $A \to B$ versus $B \to A$ test. A practical multi-hop request may require the model to recover candidates through an inverse or indirect relation and then preserve several heterogeneous filters while selecting the final list. Failure can occur at any stage: parametric recall, relation traversal, external retrieval, or post-retrieval verification.

Standard factual-QA evaluation does not describe these failures well. Exact string match cannot distinguish a clean but incomplete list from a response that mixes correct entities with constraint violations. It is also brittle to aliases, translated titles, original-language names, and transliterations. Conversely, treating every entity absent from a serialized Wikidata answer list as incorrect can penalize valid answers when the knowledge graph or stored list is incomplete. No-answer cases introduce the opposite problem: an empty response may be correct, whereas a confident list can be evidence of hallucination rather than useful recall.

We introduce MultiHopBench, a diagnostic benchmark for list-style multi-hop and multi-constraint factual QA. Its main layer is a formally grounded Wikidata core in which natural-language requests remain linked to explicit constraints, executable queries, and entity-level reference data. The release also includes a simpler control subset, curated complex examples from established QA resources, and validated zero-gold near misses. Parallel Russian formulations are provided to support evaluation of Russian-oriented systems on the same underlying tasks, but language comparison is a secondary use case rather than the central contri-

bution.[1]

The main methodological focus is the measurement of free-form answers. Instead of asking an LLM judge for one holistic score, the evaluation pipeline separates semantic annotation from arithmetic. An LLM annotator extracts and normalizes candidate entities, checks individual constraints, and assigns interpretable statuses. Deterministic code then validates the annotation, recomputes counts and rates, and aggregates them into coverage, precision, constraint, hallucination, and refusal components. This architecture preserves the flexibility needed to interpret natural-language answers while keeping the final scoring procedure auditable and reproducible.

The remainder of the paper motivates the task and positions it relative to existing QA benchmarks, describes the construction and validation of the frozen dataset, and specifies the judge–normalizer–aggregator architecture. Section 6 presents the quantitative evaluation and reporting workflow.

2 Background

2.1 Multi-Hop, Inverse Retrieval, and Constraint Intersection

Multi-hop QA usually requires traversing one or more intermediate relations before reaching an answer. Multi-constraint QA may involve a shorter relational path but a difficult intersection of filters. The two phenomena often co-occur: a system first recovers candidates through a direct, inverse, or indirect relation and then verifies attributes such as object type, location, period, category, author, participant, or numerical threshold.

This distinction matters because a model can succeed at one part of the task and fail at another. It may retrieve an entity connected to the correct person but ignore the requested time range; identify objects of the right type but from the wrong country; or preserve most filters while losing a negation. The benchmark therefore treats each request as a conjunction of explicit conditions rather than as a single opaque answer string.

2.2 List Completion and Answerability

Many benchmark items ask for several examples rather than one entity. Returning one valid item is useful but incomplete when the user requests five, while returning five candidates is not sufficient if some violate the conditions. Evaluation must therefore account for both coverage of the requested list and the purity of the entities that were returned.

The resource also includes zero-gold requests, for which the conditions have no valid intersection. These are not failed generations or records with missing fields. They are deliberately constructed near misses whose surface form is plausible but whose temporal, categorical, or relational conditions are incompatible. Correct behavior is to avoid proposing entities and, where possible, explain why no valid answer exists.

2.3 Formal Grounding and Language Variants

The Wikidata-derived core links each natural-language request to structured constraints, an executable SPARQL query, entity identifiers, labels, aliases, and metadata about answer-set completeness. This formal layer makes it possible to reproduce the reference set, inspect individual conditions, and validate a proposed entity even when it is not present in a truncated serialized list.

English is the primary surface language of the benchmark. A parallel Russian realization is stored for every record so that Russian-oriented systems can be evaluated without changing the formal task, answer set, or requested count. The paired language variants are therefore an evaluation convenience and a controlled secondary slice, rather than the defining contribution of the benchmark.

2.4 Why Exact Match Is Insufficient

Free-form answers may contain prose, aliases, original titles, transliterations, duplicated entities, hedging, partial refusals, and explanations. A single string metric collapses several substantively different outcomes: a correct but incomplete answer, a real entity that violates one condition, a substitution with a related entity, an unsupported candidate, and a fabricated entity. These errors have different implications for users and for system development.

The reference source also matters. Wikidata offers stable identifiers and executable relations, but it is not a complete oracle. A gold match is strong positive evidence; absence from a stored gold list is not, by itself, proof of error. The evaluation protocol must therefore combine gold and alias matching with constraint-level validation, uncertainty labels, and review of plausible non-gold candidates. Section 5 describes this architecture.

[1] Benchmark data, generation code, prompts, and evaluation components are available at https://github.com/matveylogee/multihop_benchmark.

3 Related Work

Multi-hop and complex QA. HotpotQA, 2WikiMultiHopQA, ComplexWebQuestions, and MuSiQue established benchmark settings for bridge, comparison, multi-document, and compositional reasoning (Yang et al., 2018; Ho et al., 2020; Talmor and Berant, 2018; Trivedi et al., 2022). These resources are central to multi-hop QA, but many instances still target one short answer and do not make list completeness, heterogeneous simultaneous constraints, or empty answer sets the primary object of evaluation. Our benchmark complements this line of work with multi-answer constraint intersections and explicit answerability probes.

Knowledge-base and multilingual QA. Wikidata provides stable entity identifiers, multilingual labels, explicit relations, and executable SPARQL queries (Vrandečić and Krötzsch, 2014). RuBQ 2.0 contributes Russian KBQA questions and Wikidata grounding, while MKQA provides aligned open-domain questions across many languages (Rybin et al., 2021; Longpre et al., 2021). KQA Pro supplies explicit compositional programs with qualifiers, comparisons, temporal operators, and set operations, and Mintaka contributes natural multilingual complex questions (Cao et al., 2022; Sen et al., 2022). We use these resources selectively: Wikidata supports the reproducible core, whereas external datasets contribute phrasing and reasoning patterns without being treated as one homogeneous source.

Model-based evaluation. LLM-based judges can interpret aliases and free-form explanations that are difficult to handle with lexical metrics, but their reliability depends on prompt design, evidence, model choice, output constraints, and calibration (Gu et al., 2024). A common risk is allowing the same model to make semantic decisions and assign an opaque holistic score. Our method restricts the LLM to atomic structured annotation. Counts, rates, group metrics, and final scores are recomputed by deterministic code, making the resulting evaluation easier to audit and reproduce.

4 Benchmark Construction

4.1 Design Principles and Subsets

The benchmark follows four design principles. First, core answers must be recoverable from an external formal source rather than from the tested model. Second, the natural-language query and its formal representation must remain linked throughout generation and evaluation. Third, heterogeneous subsets must preserve their provenance instead of being merged as if they had identical semantics. Fourth, answerable and zero-gold items must be distinguished explicitly rather than inferred from an empty field.

The frozen release is organized into four analytical subsets. The easy subset contains direct or shallow Wikidata-derived controls at levels L1–L2. The hard core contains Wikidata-derived L3–L5 requests that emphasize multi-hop retrieval, simultaneous constraints, and list completion. The external subset contributes curated complex patterns from established QA resources, while the zero-gold subset contains validated near-miss requests whose conditions have no valid intersection. Their sizes and analytical roles are summarized in Table 1.

Split	Size	Diagnostic role
Easy control	314	Direct or shallow constraints; baseline language competence
Hard core	844	List-style multi-hop and multi-constraint reasoning
External complex	81	Additional temporal, qualifier, comparison, and set-operation patterns
Zero-gold	61	Hallucination and no-answer behavior
Total	1,300	Final count after data freeze

Table 1: Benchmark subsets and their analytical roles. Source-specific metrics are reported separately.

The two Wikidata-derived answerable subsets span multiple cultural, geographic, scientific, technical, and organizational domains. The external subset combines selected KQA Pro and Mintaka examples, while the zero-gold subset is reserved for explicitly validated no-answer cases. Every record stores aligned English and Russian query fields; the Russian realization supports evaluation of Russian-oriented systems without changing the formal task.

4.2 Wikidata-Derived Core

The main layer is generated with domain-specific pipelines over Wikidata. Each domain defines relevant entity types, relations, qualifier patterns, and pools of admissible values. A generator samples a compatible combination of constraints, renders an English request and its Russian counterpart, constructs a `SELECT DISTINCT` SPARQL query,

executes it, and stores the resulting entity identifiers and labels. Generation covers cultural, geographic, scientific, technical, and organizational domains, allowing the same general evaluation framework to test different relation and constraint families.

A core record includes the two natural-language formulations, domain, difficulty level, structured constraints, requested count, gold QIDs, labels and aliases, SPARQL provenance, and flags describing answer-set completeness. Where a stored gold list is truncated or otherwise incomplete, an optional candidate-level `ASK` validator can test whether a proposed entity satisfies the same formal conditions. This distinction is important for list-style questions, where a serialized subset should not be mistaken for the complete real-world set.

Difficulty is represented by levels L1–L5. The levels are heuristic and domain-dependent rather than a universal reasoning scale. Lower levels emphasize frequent direct filters; higher levels add indirect relations, qualifiers, temporal and numeric constraints, exclusions, rare properties, and more fragile intersections. The labels are retained alongside structural features such as constraint count, relation depth, negation, and requested cardinality, which support more precise analysis than a single ordinal level.

4.3 External Complex and Zero-Gold Layers

External resources are used as curated supplements, not as a mechanical union. KQA Pro is the main donor of explicit compositional patterns, including qualifiers, temporal reasoning, comparisons, superlatives, and set operations (Cao et al., 2022). Mintaka contributes a smaller set of natural complex examples after removal of shallow bridge questions (Sen et al., 2022). RuBQ and MKQA mainly inform Russian phrasing and simpler relation patterns, while 2WikiMultiHopQA informs bridge and comparison analysis (Rybin et al., 2021; Longpre et al., 2021; Ho et al., 2020). Source identifiers and original metadata are retained so that external items can be evaluated separately from the formally generated core.

The zero-gold layer is designed around plausible near misses rather than obviously impossible requests. Examples combine mutually exclusive categories, impossible temporal orderings, historical contradictions, or relations whose intersection is empty. Empty output from a generator is not sufficient evidence for inclusion: failed queries, missing properties, and skipped records are excluded. A zero-gold item must contain an explicit answerability label, contradiction type or reason, and expected no-answer behavior, followed by formal and manual validation.

4.4 Language Realization, Enrichment, and Quality Control

English and Russian query variants are linked to a shared formal record. Automatic checks verify preservation of entities, numbers, temporal ranges, polarity, relation direction, and requested count. Manual review targets ambiguous wording, unnatural translations, and cases in which the text, constraints, and SPARQL encode different tasks.

Gold enrichment expands each target entity with Russian and English labels, aliases, original titles, and available sitelinks without replacing its QID. This improves matching across localized, transliterated, and original forms while preserving entity identity. The merge stage then converts Wikidata and external subsets into a common schema but retains source, split, answerability, provenance, and validation metadata.

Quality control combines automatic schema validation, deduplication, SPARQL and count checks, object-type validation, cross-language consistency tests, and manual sanity review. Zero-gold items, rare retrieval-dependent criteria, truncated gold sets, and externally translated examples receive stricter review because errors in these categories can distort later model comparisons.

5 Evaluation Methodology

5.1 Two-Stage Answer Evaluation

The tested model receives only the surface query and a common response-format instruction. Structured constraints, SPARQL, gold entities, completeness metadata, and the answerability label remain hidden. The raw response is stored verbatim.

Answer evaluation is deliberately split into two stages (Figure 1). First, an LLM judge converts the free-form response into a schema-conforming semantic annotation: proposed entities, entity statuses, per-constraint checks, response flags, uncertainty, and review metadata. The judge never computes counts, rates, penalties, or scores. Second, deterministic code validates the annotation against the canonical benchmark record, recomputes all derived features, and applies the fixed scoring formulas.

This separation assigns semantic interpretation

to the LLM and exact arithmetic to code. It reduces the judge's responsibility, prevents counting and formula errors from propagating into the benchmark score, and makes every numerical result reproducible and auditable. Scoring weights can also be inspected or recalibrated without changing the judge prompt or rerunning semantic annotation.

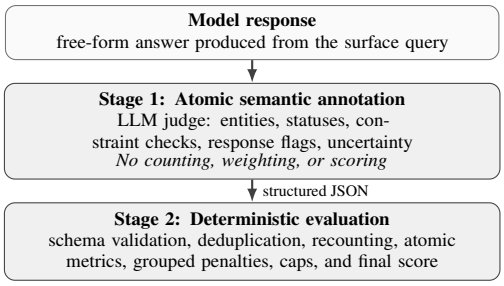

Figure 1: Two-stage answer evaluation. The LLM judge performs semantic annotation only; deterministic code validates, recomputes, and scores the resulting structure.

5.2 Atomic Semantic Annotation

For each response, the judge receives the query, requested count, answerability type, structured constraints, gold labels and aliases, gold-completeness metadata, optional trusted validation context, and the raw model response. It returns exactly one schema-conforming JSON object. The output contains the response type, one object per unique proposed entity, one check for every explicit constraint key, response-level flags, an error taxonomy, uncertainty, and a `review_needed` decision.

Each entity receives one mutually exclusive status. `gold_match` and `gold_alias_match` indicate direct reference support. `correct_by_constraints` is reserved for a non-gold entity whose key constraints are independently verified by trusted supplied evidence. `possibly_correct_not_in_gold` denotes a plausible candidate for which at least one key condition remains unresolved. Incorrect outputs are separated into `constraint_violation`, `wrong_domain`, `entity_substitution`, `unsupported_entity`, `nonexistent_hallucination`, and `ambiguous`. Repetition is represented by `mention_count`, not by a separate entity.

The judge also emits a status for every explicit constraint: `satisfied`, `violated`, `unknown`, or `not_applicable`, with severity `critical`, `important`, or `standard`. Claims made by the evaluated model are never treated as evidence. A non-gold candidate can receive full credit only when its constraints are supported by trusted context or external verification; plausibility or parametric memory alone is insufficient.

The judge records refusals, zero-gold behavior, contradictions, formatting quality, the main and secondary error types, and review reasons. It does not output counts, rates, grouped metrics, penalties, or scores. The released prompt and JSON Schema define the complete contract.

5.3 Deterministic Validation and Atomic Metrics

The judge output is treated as a semantic annotation, not trusted arithmetic. The validator enforces the JSON Schema, checks the query identifier and answerability label, requires exactly one constraint check for every structured constraint key, verifies gold references, rejects duplicate normalized entities, and tests status–constraint consistency. Invalid annotations are written to a validation log rather than silently assigned zero. Records marked as internally inconsistent are routed to review without a score; schema-valid annotations that merely require factual review can still receive a provisional score.

Let P_i be the set of unique proposed entities, $V_i \subseteq P_i$ the verified valid entities, $U_i \subseteq P_i$ the plausible unresolved entities, and k_i the requested count. The strict list-quality metrics are

$$\text{Coverage}_i = \text{clip}\left(\frac{|V_i|}{k_i}, 0, 1\right), \tag{1}$$

$$\text{Precision}_i = \frac{|V_i|}{\max(1, |P_i|)}. \tag{2}$$

The soft variants use the same denominators but replace $|V_i|$ in the numerator with $|V_i| + \alpha|U_i|$, where $\alpha = 0.5$, and clip the result to $[0, 1]$. Strict metrics therefore count only verified entities, whereas soft metrics assign partial credit to plausible unresolved candidates.

Constraint satisfaction is recomputed from the entity-level checks. Severity weights are 1.0, 0.6, and 0.3 for critical, important, and standard conditions. A satisfied check receives full credit, an unknown check receives 0.5 credit, and a violated check receives zero. The normalizer additionally derives soft precision, gold recall at the target, under- and over-answering, duplicate rate, wrong-domain and substitution rates, unsupported and nonexistent-entity rates, refusal indicators, parsing quality, and gold-completeness warnings.

5.4 Calibrated Answerable Score

Atomic metrics are aggregated into five positive groups—coverage, precision, constraint satisfaction, gold support, and format—and three principal error groups—critical error, hallucination, and refusal. Their exact within-group definitions are released with the scorer. We use the compact superscripts con, fmt, crit, hall, and ref below. For answerable items,

$$P_i^{+} = 0.30C_i^{\mathrm{cov}} + 0.25C_i^{\mathrm{prec}} + 0.30C_i^{\mathrm{con}} + 0.05C_i^{\mathrm{gold}} + 0.10C_i^{\mathrm{fmt}} + 0.11C_i^{\mathrm{poss}}, \tag{3}$$

$$P_i^{-} = 0.35E_i^{\mathrm{crit}} + 0.45E_i^{\mathrm{hall}} + 0.20E_i^{\mathrm{ref}} + 0.31E_i^{\mathrm{under}}, \tag{4}$$

$$S_i^{\mathrm{ans}} = 100\ \mathrm{clip}\big(P_i^{+}(1 - P_i^{-}), 0, 1\big)\,. \tag{5}$$

The additional possible-candidate term rewards unresolved non-gold entities only when they dominate the proposed list, their known constraints are strongly supported, and no hard error status is present. The clean-underanswer term is the product of under-answering and precision; it distinguishes a clean $2/5$ response from a clean $4/5$ response without double-penalizing contaminated lists.

Three deterministic caps encode catastrophic failure modes that are poorly represented by a purely smooth linear formula: a response consisting entirely of wrong-domain entities is capped at 10, a response consisting entirely of nonexistent entities is capped at 5, and any zero-gold response that proposes at least one entity is capped at 3. The uncapped score, applied cap, cap reason, and final score are all retained for audit.

The coefficients are fixed before the main model comparison. They were calibrated on a 16-case diagnostic suite covering complete and incomplete correct answers, verified and unresolved non-gold candidates, soft and critical constraint violations, wrong-domain outputs, hallucinations, over-answering, false refusal, and zero-gold behavior. The suite specifies qualitative ordering and target score ranges; it is a unit test for the measurement instrument, not part of the benchmark test set and not a learned reward model.

5.5 Zero-Gold Evaluation and Reporting

Zero-gold items use a separate objective because returning no entity is the desired behavior. The positive component rewards correct no-answer recognition, a compatible explanation, and parseable formatting. The penalty component measures proposed entities, critical contradictions, and inappropriate refusal behavior:

$$Z_i^{+} = 0.65C_i^{\mathrm{noans}} + 0.20C_i^{\mathrm{expl}} + 0.15C_i^{\mathrm{fmt}}, \tag{6}$$

$$Z_i^{-} = 0.65E_i^{\mathrm{zero}} + 0.25E_i^{\mathrm{crit0}} + 0.10E_i^{\mathrm{ref}}, \tag{7}$$

$$S_i^{\mathrm{zero}} = 100\ \mathrm{clip}\big(Z_i^{+}(1 - Z_i^{-}), 0, 1\big)\,. \tag{8}$$

A justified no-answer can receive full credit, while proposing entities triggers the zero-gold cap described above.

Scores are macro-averaged over queries, so records with long answer lists do not dominate the benchmark. The scorer reports per-record atomic metrics and groups, model-level provisional and official scores, and slices by domain, difficulty, subset, language, and answerability. An official score requires complete dataset coverage, no invalid annotations, and no unresolved review rows; otherwise the aggregate is explicitly marked provisional.

6 Compact Evaluation

This section reports the complete 14-cell evaluation matrix and the associated diagnostic analyses. Unavailable settings are marked as `N/A` and are never imputed.

6.1 Experimental Design and Missing-Cell Policy

The final design contains eight named systems but only 14 populated model–setting configurations. Each populated cell contains both English and Russian responses for all 1,300 benchmark records. Thus, $14 \times 2 \times 1{,}300 = 36{,}400$ responses are included. The unit of analysis is model version $\times$ setting $\times$ language $\times$ query identifier. Table 2 gives the setting definitions and the number of populated cells.

Unobserved cells are reported as N/A. They are not assigned zeros, estimated from another model, interpolated, or included in setting means. Tables and figures never assign numerical values to unpopulated cells; where the full matrix is shown, missing cells are explicitly marked as N/A. Only GPT-5.4-mini has the complete 2 × 2 search-by-reasoning design. Memory-to-reasoning contrasts are additionally available for four models; YaGPT Pro Reasoner is excluded from that paired analysis because it has no memory-only baseline.

Setting	Search	Reasoning	Cells
Memory-only	off	off	7
Search-only	on	off	1
Reasoning-only	off	native	5
Search + reasoning	on	native	1
Total			14

Table 2: Experimental design. Cell counts refer only to populated model–setting configurations.

The primary outcome is the macro-average final score from scorer v3. The supplied 95% uncertainty intervals are reported only for the primary score; intervals are not reported for the remaining diagnostics.

This compact report covers overall score, valid search/reasoning contrasts, diagnostics, language, zero-gold behavior, and list cardinality. Difficulty slices (easy–hard and L1–L5), a full error taxonomy, constraint-family and domain sensitivity, quality–cost–latency, the cinema case study, a human-judge audit, and scorer sensitivity are not measured in the supplied sparse result file. They are therefore not reported or imputed.

6.2 Overall Model–Setting Results

Table 3 and Figure 2 show the sparse result matrix. Qwen3.5-397B-A17B has the highest observed memory-only score (64.5), and DeepSeek-V3.2 has the highest observed reasoning-only score (85.7). GPT-5.4-mini is the only model evaluated in search-only and search + reasoning; the latter reaches the highest score in the report, 92.0, but it is not a cross-model combined-setting comparison.

Model	Memory-only	Search-only	Reasoning-only	Search + reasoning
GPT-5.4-mini	60.7 [59.9,61.5]	81.0 [80.4,81.6]†	78.2 [77.5,78.9]	92.0 [91.5,92.5]†
Qwen3.5-397B-A17B	**64.5 [63.7,65.3]**	N/A	84.0 [83.4,84.6]	N/A
DeepSeek-V3.2	62.1 [61.3,62.9]	N/A	**85.7 [85.1,86.3]**	N/A
Alice AI LLM 235b	57.8 [56.9,58.7]	N/A	77.5 [76.8,78.2]	N/A
YaGPT Pro Reasoner	N/A	N/A	81.2 [80.5,81.9]	N/A
YaGPT 5.1 Pro	52.4 [51.5,53.3]	N/A	N/A	N/A
Alice AI LLM 32b	37.1 [36.1,38.1]	N/A	N/A	N/A
YaGPT 5 Lite	18.7 [17.8,19.6]	N/A	N/A	N/A

Table 3: Macro-average final scores with 95% uncertainty intervals. N/A denotes an unobserved cell, not a zero. Bold marks leaders only in columns with more than one observed system. †Only evaluated system in a singleton column.

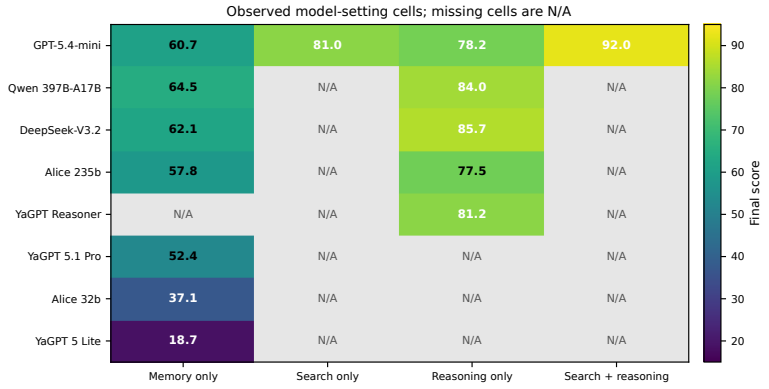

Figure 2: Score matrix for the 14 populated model–setting cells. Gray N/A cells are unobserved and are not imputed.

6.3 Search and Reasoning Contrasts

The complete factorial interpretation is restricted to GPT-5.4-mini (Table 4). Relative to memory-only, search adds 20.3 points and reasoning adds 17.5 points. Search adds 13.8 points when reasoning is already active, while reasoning adds 11.0 points when search is already active. Averaging over the other factor gives a 17.1-point search main

effect and a 14.3-point reasoning main effect. The interaction is −6.5 points, consistent with overlapping repairs or a ceiling effect: search + reasoning still exceeds search-only by 11.0 points and reasoning-only by 13.8 points.

For the four systems with paired memory-only and reasoning-only runs, reasoning improves the score by 17.5–23.6 points (Table 5 and Figure 3). The largest uplift is observed for `DeepSeek-V3.2`. The paired mean rises from 61.3 to 81.4, a 20.1-point increase. This comparison does not include the reasoning-only `YaGPT Pro Reasoner`.

Contrast	Change
Search: memory-only → search-only	+20.3
Reasoning: memory-only → reasoning-only	+17.5
Search with reasoning active	+13.8
Reasoning with search active	+11.0
Search main effect	+17.1
Reasoning main effect	+14.3
Search × reasoning interaction	−6.5

Table 4: Factorial contrasts for `GPT-5.4-mini`, the only complete 2 × 2 design.

Model	Memory-only	Reasoning-only	Δ
`GPT-5.4-mini`	60.7	78.2	+17.5
`Qwen3.5-397B-A17B`	64.5	84.0	+19.5
`DeepSeek-V3.2`	62.1	85.7	**+23.6**
`Alice AI LLM 235b`	57.8	77.5	+19.7
Paired mean	61.3	81.4	**+20.1**

Table 5: Reasoning uplift for models with both endpoints. Unpaired systems are excluded rather than imputed.

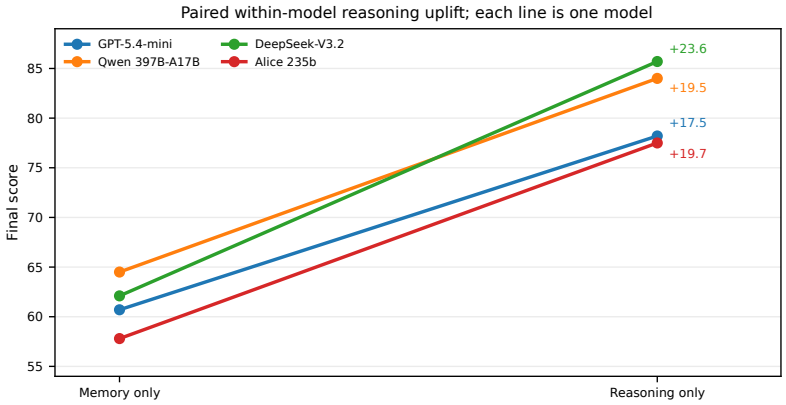

Figure 3: Paired memory-only-to-reasoning-only changes. Only the four models with both endpoints are drawn.

6.4 Diagnostic Metrics

Figure 4 reports diagnostics for every populated configuration. The complete `GPT-5.4-mini` design illustrates the different mechanisms: search primarily raises coverage and reduces under-answering, while reasoning yields stronger constraint satisfaction and correct no-answer recognition than memory-only. Their combination has the best diagnostic profile in that model, with 0.949 coverage, 0.968 constraint satisfaction, 0.043 under-answering, and 0.008 hallucination or unsupported-entity rate.

For reasoning-only runs, `DeepSeek-V3.2` has the highest coverage (0.861), precision (0.918), and constraint satisfaction (0.938), while `YaGPT Pro Reasoner` has the highest correct no-answer rate (0.941). The low score of `YaGPT 5 Lite` is associated with very low coverage (0.183), substantial under-answering (0.799), and a high unsupported-entity rate (0.258), rather than uniformly incorrect entities.

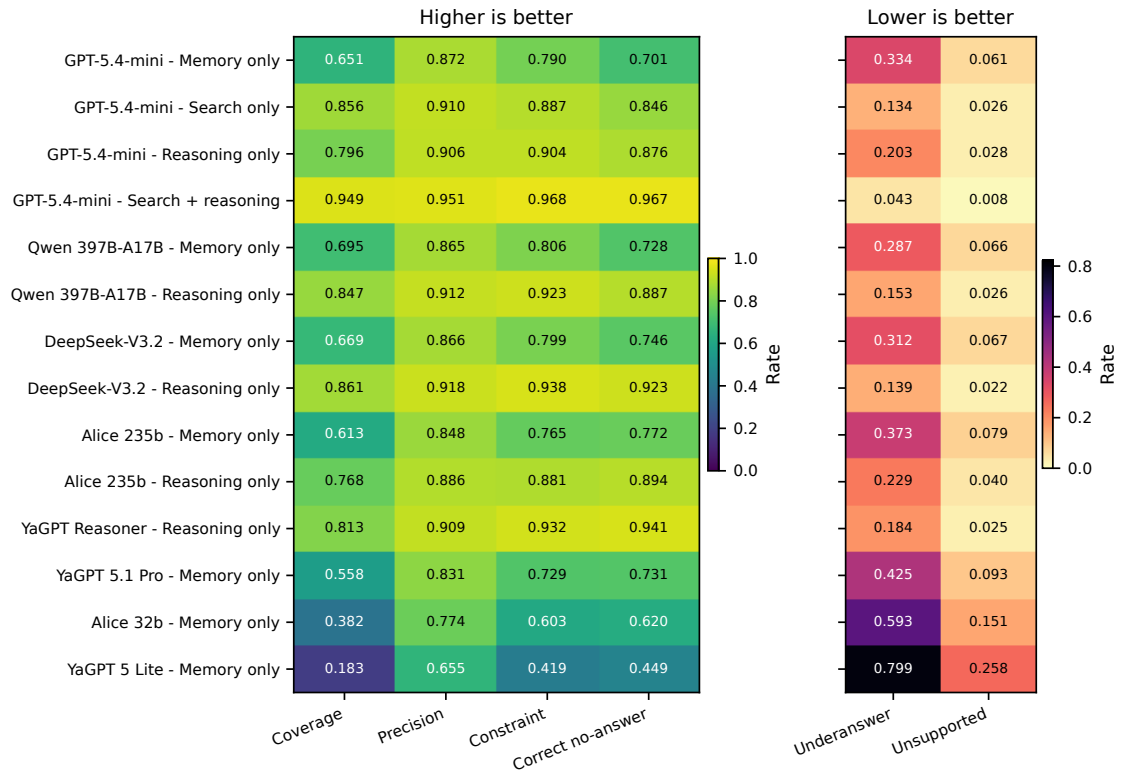

Figure 4: Diagnostic heatmaps for all observed configurations. Every plotted row corresponds to a populated cell; no missing cell is imputed.

6.5 Language Analysis

The English and Russian means underlying Figure 5 average to the overall scores in Table 3; exact rows remain in the canonical result file distributed with the sources. Language gaps are small for `GPT-5.4-mini`, `Qwen3.5-397B-A17B`, and `DeepSeek-V3.2`. The Russian advantage is larger for the Alice and YaGPT configurations and increases to 11.0 points for `YaGPT 5 Lite`. Because model family and setting availability are confounded, these configuration-level gaps should not be attributed causally to architecture or reasoning.

6.6 Zero-Gold Answerability

Zero-gold results are shown in Table 6 and Figure 6. Within the complete `GPT-5.4-mini` design, correct no-answer recognition increases from 70.1% in memory-only to 96.7% with search + reasoning, while the entity-proposal rate falls from 20.5% to 1.6%. Among reasoning-only observations, `YaGPT Pro Reasoner` has the highest correct no-answer rate (94.1%), followed by `DeepSeek-V3.2` (92.3%). `YaGPT 5 Lite` proposes an entity on 47.5% of zero-gold cases.

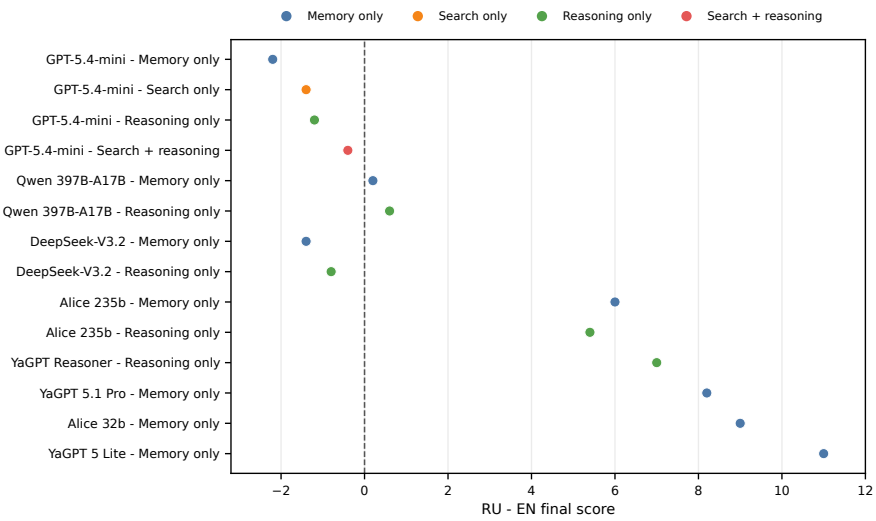

Figure 5: Russian-minus-English score gaps by observed configuration. No unobserved configuration appears in the plot.

Model	Setting	Zero-gold score	Correct no-answer	Entity proposal
`GPT-5.4-mini`	memory-only	68.4	70.1%	20.5%
	search-only	83.2	84.6%	9.8%
	reasoning-only	86.8	87.6%	7.4%
	search + reasoning	**96.0**	**96.7%**	**1.6%**
`Qwen3.5-397B-A17B`	memory-only	71.6	72.8%	18.0%
	reasoning-only	88.0	88.7%	6.6%
`DeepSeek-V3.2`	memory-only	73.3	74.6%	16.4%
	reasoning-only	91.5	92.3%	4.1%
`Alice AI LLM 235b`	memory-only	76.5	77.2%	13.9%
	reasoning-only	88.8	89.4%	5.7%
`YaGPT Pro Reasoner`	reasoning-only	93.4	94.1%	2.5%
`YaGPT 5.1 Pro`	memory-only	72.1	73.1%	17.2%
`Alice AI LLM 32b`	memory-only	60.1	62.0%	29.5%
`YaGPT 5 Lite`	memory-only	42.0	44.9%	47.5%

Table 6: Zero-gold behavior for observed configurations. Entity proposal is the share of zero-gold records on which at least one entity is proposed.

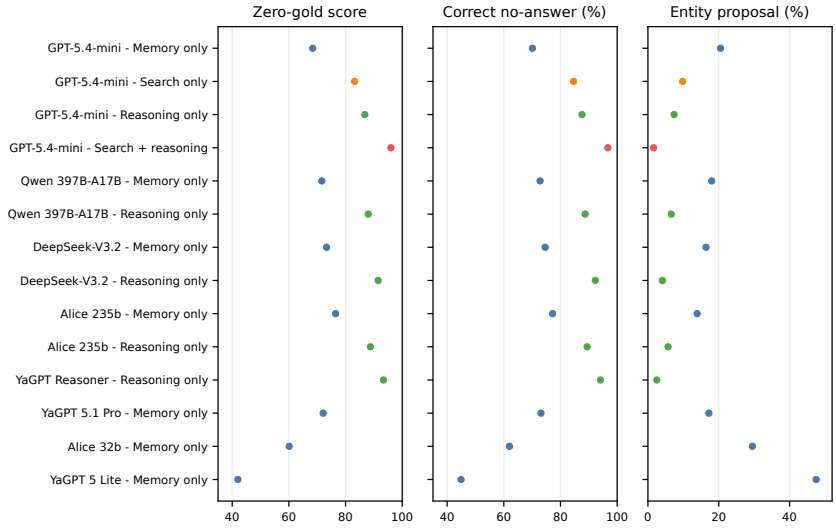

Figure 6: Zero-gold score, correct no-answer rate, and entity-proposal rate for the 14 observed configurations.

6.7 List Completeness

Strict coverage falls with requested list size for every observed configuration (Table 7 and Figure 7). For requests with more than five entities, `DeepSeek-V3.2` reasoning-only has the highest observed strict coverage at 0.85, followed by `Qwen3.5-397B-A17B` reasoning-only at 0.82 and `GPT-5.4-mini` search-only at 0.80. The `GPT-5.4-mini` search + reasoning setting reaches 0.90. In contrast, `YaGPT 5 Lite` memory-only falls to 0.02, explaining much of its low aggregate score through incomplete lists and simultaneous-constraint failures.

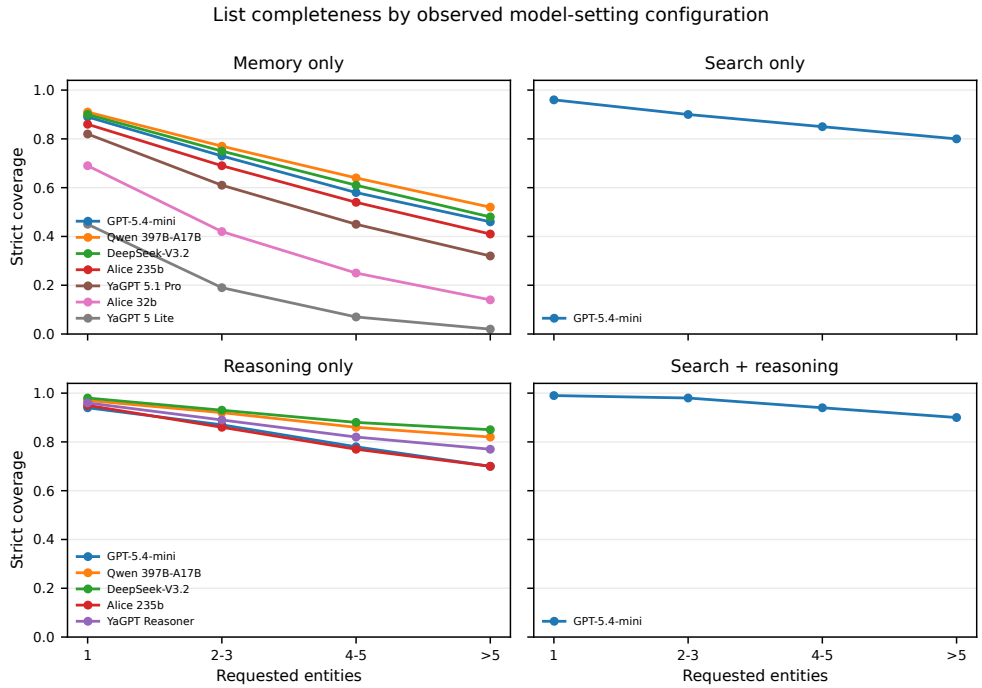

Figure 7: Strict-coverage curves grouped by setting. Only available model–setting configurations are drawn; all connected endpoints within a curve are observed list-size bins.

Model	Setting	Requested 1	Requested 2–3	Requested 4–5	Requested >5
`GPT-5.4-mini`	memory-only	.89	.73	.58	.46
	search-only	.96	.90	.85	.80
	reasoning-only	.94	.87	.78	.70
	search + reasoning	**.99**	**.98**	**.94**	**.90**
`Qwen3.5-397B-A17B`	memory-only	.91	.77	.64	.52
	reasoning-only	.97	.92	.86	.82
`DeepSeek-V3.2`	memory-only	.90	.75	.61	.48
	reasoning-only	.98	.93	.88	.85
`Alice AI LLM 235b`	memory-only	.86	.69	.54	.41
	reasoning-only	.95	.86	.77	.70
`YaGPT Pro Reasoner`	reasoning-only	.96	.89	.82	.77
`YaGPT 5.1 Pro`	memory-only	.82	.61	.45	.32
`Alice AI LLM 32b`	memory-only	.69	.42	.25	.14
`YaGPT 5 Lite`	memory-only	.45	.19	.07	.02

Table 7: Strict coverage by requested list size for all observed configurations. Every row corresponds to an available configuration; no absent mode is filled in.

7 Conclusion

We presented a formally grounded benchmark for multi-hop and multi-constraint factual QA, with English requests as the primary evaluation surface and parallel Russian variants for Russian-oriented systems. The data pipeline combines a reproducible Wikidata core with curated external reasoning patterns, parallel language realizations, multilingual label and alias enrichment, source-preserving merge, and a validated zero-gold layer. The evaluation methodology separates an LLM-based semantic annotator from deterministic normalization and score aggregation, enabling analysis of incomplete coverage, missed constraints, substitutions, wrong-domain entities, unsupported claims, and false refusals rather than reducing every response to exact match.

The compact report contains eight named models, 14 populated model–setting configurations, and 36,400 responses. Unavailable modes are shown as `N/A` throughout, omitted from plots, and never imputed. `GPT-5.4-mini`, the only complete search-by-reasoning design, improves from 60.7 in memory-only to 81.0 in search-only, 78.2 in reasoning-only, and 92.0 with search + reasoning. In the four valid paired comparisons, reasoning adds 17.5–23.6 points, with the largest uplift for `DeepSeek-V3.2`. Qwen leads the observed memory-only column, DeepSeek leads the observed reasoning-only column, and no cross-model conclusion is made for search-only or search + reasoning because each contains only one evaluated system. Diagnostic, language, zero-gold, and list-completeness views reveal behavior hidden by the aggregate score. Unmeasured difficulty, error-taxonomy, constraint-family, domain, cost, case-study, human-audit, and scorer-sensitivity slices are deliberately omitted rather than imputed.

Limitations

Wikidata provides a reproducible formal reference but is not a complete or error-free oracle. Relevant properties may be missing, labels may be ambiguous, and later graph updates can alter answer sets. QIDs, aliases, candidate validators, completeness flags, partial credit for unresolved non-gold candidates, and manual review reduce these risks but do not eliminate them.

The external layer has weaker common formal grounding than the Wikidata core. Its quality depends on the source dataset, filtering, translation, and manual selection. It is therefore reported separately. Likewise, Russian and English formulations may differ in naturalness or ambiguity despite sharing a formal record; measured language effects can include residual realization artifacts.

L1–L5 difficulty labels are heuristic and partly domain-specific. They should be analyzed together with structural features such as constraint count, relation depth, negation, rarity, and answer cardinality; the current evaluation does not include difficulty-slice results. Zero-gold items are especially sensitive to false emptiness caused by incomplete knowledge sources; uncertain cases must be excluded or routed to review rather than forced into the no-answer subset.

The scoring function is an interpretable calibrated specification, not a learned or statistically optimal reward model. Its coefficients were checked on a finite diagnostic suite and may not capture every edge case. The LLM judge can also make entity extraction, alias resolution, or fact-verification errors. The schema, deterministic cross-checks, uncertainty labels, review queue, and calibration suite make these errors visible but cannot eliminate them. Finally, public facts may have appeared in model pretraining, and search-enabled systems have different information access from closed-book models; the benchmark measures practical answer quality rather than contamination-free reasoning ability.

The model–setting design is also sparse and unbalanced. Only `GPT-5.4-mini` has the complete search-by-reasoning factorial design, only four systems have paired memory-only and reasoning-only observations, and search-only and search + reasoning contain one system each. Consequently, setting availability is confounded with model identity. The report uses `N/A`, excludes unpaired systems from contrasts, and omits missing cells from plots, but those safeguards cannot recover the absent comparisons. In particular, the 92.0 combined score cannot support a cross-model combined-setting ranking, and the language and diagnostic patterns cannot be cleanly separated from the available model mix.

Ethical Considerations

The benchmark contains factual questions about public entities and is not designed to infer sensitive personal attributes. Dataset releases should preserve source licenses and attribution, document

retrieval dates and source-specific provenance, and distinguish generated, translated, and externally curated records. Model responses, judge annotations, and aggregate scores should not be interpreted as complete measures of truthfulness, safety, or general language competence. When proprietary systems are evaluated, the public paper should retain a reproducible comparison based on accessible models and clearly separate public quantitative results from non-disclosed industrial validation.

Acknowledgments

The author would like to thank Irina Lyalikova of the YandexGPT Pretraining Analytics team for helpful clarifications and suggestions.

References

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. 2023. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*.

Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyiu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, and Hanwang Zhang. 2022. KQA Pro: A dataset with explicit compositional programs for complex question answering over knowledge base. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 6101–6119.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2024. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625.

Shayne Longpre, Yi Lu, and Joachim Daiber. 2021. MKQA: A linguistically diverse benchmark for multilingual open domain question answering. *Transactions of the Association for Computational Linguistics*, 9:1389–1406.

Ivan Rybin, Vladislav Korablinov, Valentin Biryukov, and Mikhail Burtsev. 2021. RuBQ 2.0: An innovated russian question answering dataset. In *Artificial Intelligence and Natural Language*, Communications in Computer and Information Science.

Priyanka Sen, Alham Fikri Aji, and Amir Saffari. 2022. Mintaka: A complex, natural, and multilingual dataset for end-to-end question answering. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 1604–1619.

Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 641–651.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A free collaborative knowledge base. *Communications of the ACM*, 57(10):78–85.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.